Machine Learning for Managers

Decision Tree Algorithm & Classifier Evaluation
(Confusion Matrix, Precision / Recall / F1, ROC–AUC)

Plain-English Study Guide with Worked Examples

Built on Sessions 7 & 8 (Prof. Sangeeta Shah Bharadwaj)
Prepared for Kishore Kunal · Roll No. 38BM21
MDI Gurgaon · PGDM-BM/BA

Part A — Decision Tree Algorithm

Read this like a story: plain-English idea first, then the proper term, then the analogy, then the numbers worked all the way through. Yellow boxes = testable. Red boxes = traps. Blue boxes = analogies.

A.1 The plain-English idea

Imagine deciding whether to wait for a table at a busy restaurant. You don't run an equation — you ask a chain of yes/no questions: *"Are people already waiting? No? Then I'll wait."* You funnel yourself down a path of questions until you reach a decision. That flowchart of questions **is a decision tree.**

Proper term: A **Decision Tree** is a *supervised* learning algorithm shaped like an upside-down tree. You feed it labelled examples, and it *learns* the best sequence of questions so that, for a new situation, it can predict the answer.

> **ANALOGY** — It's a game of *Twenty Questions*. Each question you ask should slice the remaining possibilities as cleanly as possible.

Contrast with linear regression: linear regression fits *one straight line* through all the data at once. A decision tree instead **carves the data into rectangular regions**, asking about one variable at a time. Hold onto that contrast — it is testable.

A.2 Anatomy of a tree

- **Root node** — the top box; the *first* question; sees 100% of the data. There is exactly one.
- **Internal nodes** (a.k.a. *decision nodes*) — middle boxes; each asks a further question and splits again.
- **Leaf nodes** (a.k.a. *terminal nodes*) — bottom boxes; endpoints that give the final answer. No children.
- **Branches** — the lines connecting nodes. A parent node's splits produce **child** nodes.

> **EXAM HOOK** — Root = first split, sees all data, only one. Leaf = terminal, gives the prediction, no children. The paired synonyms *internal/decision node* and *terminal/leaf node* are favourite quiz items.

A.3 Classification tree vs Regression tree

Sometimes you predict a *category* (Wait? Yes/No), sometimes a *number* (life expectancy).

- Target is **discrete/categorical → Classification Tree**
- Target is **continuous → Regression Tree**

A tree can do *both* — same shape, different splitting criterion.

> **EXAM HOOK** — Decision trees handle *both* discrete and continuous targets. Classification → discrete; Regression → continuous.

A.4 The running case: the restaurant dataset

Classic Russell & Norvig data: **12 customers, 10 attributes**, target = **WillWait (True/False)**. Attributes: *Alternate, Bar, Fri/Sat, Hungry, Patrons (None/Some/Full), Price ($/$$/$$$), Raining, Reservation, Type (French/Italian/Thai/Burger), WaitEstimate (0–10/10–30/30–60/>60 min).*

Of the 12 examples, **6 said Wait (positive) and 6 said Don't-Wait (negative)** — a balanced 50/50 start. Every calculation flows from this 6/6.

A.5 The central question — where to split?

Intuition: a good question *separates the answers cleanly*. Think of sorting a mixed bag of red and green marbles. A great question drops all greens in one bucket, all reds in another — each becomes "pure". A useless question leaves both buckets half-and-half.

So we need a number for "how mixed is this bucket?" — a **measure of impurity**. The two famous ones: **Entropy** and the **Gini Impurity Index**.

A.6 Entropy — measuring messiness

Plain English: Entropy measures **randomness / disorder**. All-one-colour → zero disorder. Perfect 50/50 → maximum disorder.

> **ANALOGY** — Entropy is the "surprise" in a coin. Two-headed coin (always heads) → entropy 0. Fair coin (50/50) → entropy 1.

Key numbers (memorise):

- Entropy lies **between 0 and 1** (two-class problem).
- Entropy **0** = perfectly pure (what we want in a leaf).
- Entropy **1** = maximum randomness (perfect 50/50).
- **Higher entropy = higher impurity.**

> **TRAP — deck wording** One slide says "Entropy of 0.5 means highest randomness." That is misleading. For two-class entropy, the maximum (50/50) equals **1**, not 0.5. The **0.5 is the probability p on the x-axis**; at p = 0.5 the entropy *peaks at 1*. (The Gini curve *does* peak at 0.5 — likely the source of the mix-up.)

$$Entropy = \sum_{i=1}^{n} -p(c_i)\log_2\big(p(c_i)\big)$$

In words: for each class, take its proportion \(p\), multiply by \(\log_2\) of that proportion, add a minus sign, sum across classes.

Worked example — a 50/50 bucket (3 pos, 3 neg)

\(p(\text{pos}) = 3/6 = 0.5,\; p(\text{neg}) = 0.5\). Since \(\log_2(0.5) = -1\):

$$Entropy = -0.5(-1) - 0.5(-1) = 0.5 + 0.5 = 1$$

A 50/50 split gives **entropy = 1** — the maximum.

Worked example — a pure bucket (4 pos, 0 neg)

\(p(\text{pos}) = 1,\; \log_2(1) = 0\), empty class contributes 0:

$$Entropy = -1 \times 0 = 0$$

A.7 Information Gain — did the split help?

Plain English: Information Gain = how much entropy *dropped* because of the split. Start with the parent's messiness; subtract the (weighted) messiness of the children. Bigger drop = better question.

> **ANALOGY** — The "aha!" of a good Twenty-Questions move. "Is it an animal?" splits possibilities hugely (high gain). "Is it spelled with a Q?" barely narrows anything (near-zero gain).

$$\text{Information Gain} = \text{Entropy(parent)} - \big[\text{weighted average Entropy(children)}\big]$$

"Weighted" matters: a child holding 17 of 30 items counts more than one holding 13. Weight each child's entropy by its *fraction of the data*.

Full worked example — the deck's 30-instance split

Population of **30**, starting **14 positive / 16 negative**, splits into:

- **Child 1:** 17 instances → 13 pos, 4 neg
- **Child 2:** 13 instances → 1 pos, 12 neg

Step 1 — Parent entropy (14 pos, 16 neg / 30):

$$-\Big(\tfrac{14}{30}\log_2\tfrac{14}{30}\Big) - \Big(\tfrac{16}{30}\log_2\tfrac{16}{30}\Big) = 0.996$$

Nearly 1 — sensible, since 14 vs 16 is almost 50/50.

Step 2 — Child 1 entropy (13 pos, 4 neg / 17):

$$-\Big(\tfrac{13}{17}\log_2\tfrac{13}{17}\Big) - \Big(\tfrac{4}{17}\log_2\tfrac{4}{17}\Big) = 0.787$$

Step 3 — Child 2 entropy (1 pos, 12 neg / 13):

$$-\Big(\tfrac{1}{13}\log_2\tfrac{1}{13}\Big) - \Big(\tfrac{12}{13}\log_2\tfrac{12}{13}\Big) = 0.391$$

Child 2 is much purer (0.391) — almost all negatives.

Step 4 — Weighted average entropy of children (Child 1 = 17/30, Child 2 = 13/30):

$$\Big(\tfrac{17}{30}\times 0.787\Big) + \Big(\tfrac{13}{30}\times 0.391\Big) = 0.446 + 0.169 = 0.615$$

Step 5 — Information Gain:

$$0.996 - 0.615 = 0.38$$

The algorithm computes this for *every* attribute and picks the one with the **highest gain**, then repeats on each child.

> **EXAM HOOK — the four-step recipe**
>
> 1. Calculate entropy of the parent (the class).
> 2. Calculate entropy of each attribute *as if* you split on it.
> 3. Information Gain = parent entropy − weighted entropy of the attribute.
> 4. **Pick the attribute with maximum Information Gain.**

A.8 Gini Impurity — the faster alternative

Plain English: Gini asks — *if I pull one item from this bucket and label it by randomly guessing according to the bucket's colour proportions, how often do I guess wrong?* Pure bucket → never wrong → Gini = 0. Messy bucket → often wrong → Gini high.

$$Gini = 1 - \sum_{i=1}^{n} p^2(c_i)$$

Take each class proportion, square it, sum, subtract from 1.

Why Gini if we have entropy? Speed — squaring is cheaper than logarithms. They usually pick nearly the same splits. Algorithm families:

- **CART** (Classification & Regression Trees) → uses **Gini**
- **CHAID** (Chi-square Automatic Interaction Detection) → uses a chi-square test
- Entropy / Information Gain → ID3 / C4.5 family

> **EXAM HOOK — split-criteria one-liners**
>
> - **Gini Index** — favours *larger* partitions; uses *squared* probabilities.
> - **Entropy / Information Gain** — favours *smaller* partitions; uses *base-2 log*.
> - **Gain Ratio** — normalises Information Gain to *penalise many small splits*.
> - **ANOVA** — used in *regression* trees; *minimises variance*.

Worked example — Gini for Patrons vs Type (deck numbers)

Compare two candidate roots, both from the 6-pos/6-neg parent.

Candidate A — split on Patrons? (None / Some / Full)

Branch	Examples	Pos	Neg	Purity
None	2	0	2	pure
Some	4	4	0	pure
Full	6	2	4	mixed

Gini of each child:

- $GI(\text{None}) = 0$, $GI(\text{Some}) = 0$
- $GI(\text{Full}) = 1 - \big[(2/6)^2 + (4/6)^2\big] = 1 - [0.111 + 0.444] = 0.44$

Weighted Gini of the split (only Full is impure, holding 6/12):

$$GI(\text{Node}) = \tfrac{6}{12}\times 0.44 = \tfrac{1}{2}\times\tfrac{16}{36} = 0.22$$

Gini gain: parent $GI = 2\times\tfrac{6}{12}\times\tfrac{6}{12} = 0.5$, so gain $= 0.5 - 0.22 = \mathbf{0.28}$.

> **WATCH** The deck writes 0.22 for the "Full" line — that is the *weighted contribution*, whereas the raw Gini of the Full node itself is **0.44**. Don't confuse the node's own Gini with its weighted contribution.

Candidate B — split on Type? (French / Italian / Thai / Burger)

Branch	Pos	Neg	Gini
French	1	1	0.5
Italian	1	1	0.5
Thai	2	2	0.5
Burger	2	2	0.5

Every branch is a 50/50 mess — Type tells us nothing:

$$GI(\text{Node}) = \Big(\tfrac{2}{12}+\tfrac{2}{12}+\tfrac{4}{12}+\tfrac{4}{12}\Big)\times 0.5 = 0.5 \quad\Rightarrow\quad \text{Gain} = 0.5 - 0.5 = 0$$

Verdict: Patrons gain 0.28 vs Type gain 0 → the tree picks **Patrons** as root. Deck phrasing: choose **Min(0.22, 0.5)** impurity = **Max(0.28, 0.00)** gain.

> **EXAM HOOK** — Two equivalent framings that always agree: pick **minimum impurity (Gini)** OR **maximum gain**. Lowest impurity = highest gain = best split.

A.9 Regression trees — splitting on numbers

Plain English: No more red/green marbles. Now measure **spread**. A good split makes groups whose numbers are *close together*. The tool for spread is the one from OLS: **squared errors.**

> **ANALOGY** — Sort students into two rooms so that within each room everyone scored similarly. Success = little variation *within* each room.

$$RSS = \sum_{j=1}^{J}\sum_{i \in R_j}(y_i - \hat{y}_{R_j})^2$$

For each region R_j, the prediction $\hat{y}_{R_j}$ is the **average** of the y-values in it. Sum the squared distances from that average. Lowest total wins. (Also called **SSE** / **variance reduction**.)

> **EXAM HOOK** — Classification trees minimise **Gini/Entropy**; regression trees minimise **RSS/SSE/variance**. A regression tree's prediction for a region = **mean of training y-values in that region.**

Worked example — income vs life-expectancy (deck case)

Annual Income (₹ lakhs)	Life Expectancy (yrs)
6.5	75
7.0	85
8.0	80
8.5	67

Step 1 — Sort X ascending (done).

Step 2 — Candidate thresholds = midpoints: 6.75, 7.5, 8.25.

Try split at X ≤ 8.25 → left {6.5, 7.0, 8.0}, right {8.5}

- Left avg = (75+85+80)/3 = 80; Right = 67
- SE = $(75-80)^2+(85-80)^2+(80-80)^2+(67-67)^2 = 25+25+0+0$
- **SSE = 50**

Try split at X ≤ 6.75 → left {6.5}, right {7.0, 8.0, 8.5}

- Left = 75; Right avg = (85+80+67)/3 = 77.33
- SE = $(0+59.29+7.29+106.09$
- **SSE = 172.67**

Try split at X ≤ 7.5 → left {6.5, 7.0}, right {8.0, 8.5}

- Left avg = 80; Right avg = 73.5
- SE = $(25+25+42.25+42.25$
- **SSE = 134.5**

Verdict: 50 vs 172.67 vs 134.5 → split at **X ≤ 8.25 wins** (lowest SSE = 50). Becomes the root.

Root node MSE

Before splitting, node predicts overall average (75+85+80+67)/4 = **76.75**. MSE = SSE/n:

$$\frac{(75-76.75)^2+(85-76.75)^2+(80-76.75)^2+(67-76.75)^2}{4} = \frac{176.75}{4} = 44.19$$

matching the deck's `mse = 44.188`. After splitting, the {6.5,7,8} leaf (mean 80) has MSE (25+25+0)/3 = **16.67**; the lone {8.5} leaf has MSE 0. The tree keeps splitting each impure leaf by the same midpoint procedure until every leaf is pure — a perfect segue into overfitting.

A.10 Overfitting — the tree's fatal temptation

Plain English: Unchecked, a tree keeps splitting until *every training example sits in its own perfect leaf*. It **memorises** the data — 100% training accuracy, but it learned the *noise*, not the *pattern*. On new data it flails.

This is the **high-variance** end of the **bias–variance trade-off**: a too-deep tree = high variance = overfitting. The deck's "Really Overfit" slide (a monstrously tangled tree) makes the point.

> **ANALOGY** — The student who memorises last year's exact answers instead of understanding concepts: aces the identical paper, fails the moment questions are reworded.

The cure — pruning

Pre-pruning (stop early / "early stopping")

Halt growth *before* it goes wild. Three knobs (R's `rpart`):

- **minsplit** — min observations a *parent* must have to be allowed to split. R default = **20**. Fewer than 20 → make it a leaf.
- **minbucket** — min observations allowed in a *leaf*. A split creating a leaf smaller than this is rejected (often set to minsplit/3).
- **maxdepth** — hard ceiling on tree height. Deck example = 5; R default = **30** (beyond ~30 misbehaves on 32-bit machines).

Also framed as three questions: *Minimum Parent Node Size, Minimum Split Size, Cost Complexity* (is another node worth it?).

Post-pruning (grow full, then trim)

Let the tree grow completely, then cut branches that don't earn their keep. Two moves: **remove redundant rules** and **cut to a certain depth**. Crucially, use a separate **testing/validation set**: if removing a branch improves or doesn't hurt test performance, it was overfitting — cut it.

> **EXAM HOOK — parameters & defaults**
>
> - **minsplit** = min obs in a *parent* to allow a split (R default **20**)
> - **minbucket** = min obs in a *leaf*
> - **maxdepth** = max height (R default **30**, deck example **5**)
> - **Pre-pruning** = stop early during growth; **Post-pruning** = grow full then trim with a test set.
> - Classic trip-up: minsplit governs the *parent*; minbucket governs the *leaf*.

A.11 The headline weakness — only straight cuts

Plain English: A tree can only slice with cuts **parallel to the axes** — one variable at a time, a horizontal or vertical line. It cannot draw a diagonal in one stroke. If the true boundary is a smooth diagonal or curve (e.g. classes in an ellipse), the tree approximates it with a clumsy **staircase** of many little steps.

Deck contrast: on *linearly separable Iris*, a shallow tree (Petal.Length < 2.5? then Petal.Width < 1.8?) nails it. On *elliptical, non-linear data*, the tree explodes into an ugly staircase — while SVM draws the curve cleanly.

> **EXAM HOOK** — The biggest drawback: a Decision Tree needs **linearly separable** data — only straight lines, one variable at a time (**axis-parallel** splits). This is *why* we move to ensembles.

A.12 The bridge to ensembles

One tree is a "weak learner": twitchy, easily overfit, only straight cuts. Combine many trees and their errors cancel.

Random Forest (plain English)

Grow *many* trees, each on a random sample of the data, each considering only a random subset of features at each split. Then **vote** (classification) or **average** (regression). No single tree's quirks dominate.

- **Bagging (Bootstrap Aggregating):** draw random samples *with replacement*, train a tree on each, aggregate. Deck's 4 steps: (1) random sample n ≤ N with replacement; (2) grow a tree using *all* predictors; (3) repeat for all samples; (4) aggregate by **majority vote** (classification) or **mean** (regression).
- **Random feature selection:** the twist that makes a "forest" more than bagging — only a random subset of predictors considered at each split.

> **EXAM HOOK — Bagging vs Random Forest**
>
> - **Bagging** uses the **full set** of predictors at each split.
> - **Random Forest** uses a **random subset** of predictors at each split.
> - Optimal predictors to try: **m = √K** (classification), **m = K/3** (regression), K = total predictors.
> - Therefore **Bagging = the special case of Random Forest where m = K.**

Out-of-Bag (OOB) error: records left out of a tree's with-replacement sample act as a free validation set — measured via classification accuracy or regression R^2 & RMSE. Random Forest benefits: *avoids overfitting, handles many features, aids feature-importance selection.*

Gradient Boosting (next-topic sneak peek): Random Forest builds trees *independently, in parallel*, then votes. Boosting builds trees *sequentially*, each new tree fixing the previous trees' mistakes.

Bagging / RF = parallel + independent + vote | Boosting = sequential + error-correcting

Part B — Judging a Classifier: The Confusion Matrix & Friends

Once a classifier makes predictions, how good is it? "Accuracy" alone can lie. This part builds the full evaluation toolkit — the confusion matrix, then the four metrics that come out of it, then how to extend everything to more than two classes, then ROC/AUC.

B.1 The four building blocks

Plain English: When a yes/no classifier makes a call, there are exactly four outcomes. Suppose we're detecting cats (cat = "positive"):

- **True Positive (TP)** — predicted cat, and it *is* a cat. Correct.
- **True Negative (TN)** — predicted not-cat, and it *isn't*. Correct.
- **False Positive (FP)** — predicted cat, but it's *not* (a dog). A "false alarm." (Type I error)
- **False Negative (FN)** — predicted not-cat, but it *was* a cat. A "miss." (Type II error)

> **ANALOGY** — A smoke alarm. It beeps with no fire = **false positive** (annoying false alarm). It stays silent during a real fire = **false negative** (dangerous miss). Which error is worse depends entirely on context — and that's the whole point of looking beyond accuracy.

Arrange the four counts in a 2×2 grid — the **Confusion Matrix**:

	Predicted Positive	**Predicted Negative**
Actual Positive	**TP**	FN
Actual Negative	FP	**TN**

> **EXAM HOOK** — FP = Type I error (false alarm). FN = Type II error (miss). The diagonal (TP, TN) = correct; the off-diagonal (FP, FN) = errors. Know the memory-jog interpretations: "Recall 70% means 3 of every 10 real cats are missed and 7 correctly found"; "Precision 60% means 4 of every 10 predicted cats are actually dogs."

B.2 The four metrics

Accuracy — the headline (and why it can mislead)

Plain English: of all predictions, what fraction were correct?

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Why it lies: if 99% of emails are legit and 1% spam, a lazy model that says "never spam" scores 99% accuracy — while catching zero spam. That's why we need precision and recall.

Precision — "when it says yes, how often is it right?"

Plain English: of everything the model *flagged as positive*, what fraction truly was? Precision cares about **false alarms**.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{\text{true positives}}{\text{predicted positives}}$$

Recall (Sensitivity) — "of all the real ones, how many did it catch?"

Plain English: of everything that *actually was positive*, what fraction did the model find? Recall cares about **misses**.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{\text{true positives}}{\text{all actual positives}}$$

ANALOGY — precision vs recall tension Fishing with a net. A *tiny* net catches only fish you're sure of → high precision, but you miss many (low recall). A *giant* net scoops everything → high recall, but lots of junk (low precision). Improving one often hurts the other.

EXAM HOOK — which matters when? Cancer screening → prioritise **Recall** (a miss is deadly; a false alarm just means more tests). Spam filter / criminal conviction → prioritise **Precision** (a false positive is very costly). Also remember the extra deck definitions: **Specificity** = TN/(TN+FP) — of all real negatives, how many were correctly called negative.

F1-Score — one number balancing both

Plain English: since precision and recall pull against each other, F1 combines them into a single score using the **harmonic mean** (which punishes imbalance — you can't fake it by maxing just one).

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

WHY HARMONIC, NOT ARITHMETIC If Precision = 1.0 and Recall = 0.0, the ordinary average is 0.5 (flattering). The harmonic mean gives 0 — correctly telling you the model is useless. F1 rewards *balance*.

B.3 Fully worked 2×2 example — the cat/dog matrix (deck)

The deck's confusion matrix, with Dog as the positive class:

	Predicted Dog	Predicted Not-Dog
Actual Dog	**TP = 5**	FN = 1
Actual Not-Dog	FP = 1	**TN = 3**

Totals: 6 actual dogs, 4 actual not-dogs, 10 samples.

Accuracy:

$$\frac{TP + TN}{\text{total}} = \frac{5 + 3}{10} = \frac{8}{10} = 0.80$$

Precision (Dog):

$$\frac{TP}{TP+FP} = \frac{5}{5+1} = \frac{5}{6} = 0.83$$

Recall (Dog):

$$\frac{TP}{TP+FN} = \frac{5}{5+1} = \frac{5}{6} = 0.83$$

F1 (Dog):

$$2\times\frac{0.83\times 0.83}{0.83+0.83} = 0.83$$

These match the deck's report: Dog row 0.83 / 0.83 / 0.83, and the Not-Dog class gives 0.75 / 0.75 / 0.75. The overall **accuracy = 0.80**, **macro avg = 0.79**, **weighted avg = 0.80**.

B.4 More than two classes — the one-vs-all trick

Plain English: real problems often have many classes (digit 1/2/3/4…). To reuse our 2×2 metrics, we take *one class at a time* as "positive" and lump *all the rest* as "negative." That collapses a big matrix into a familiar 2×2 — repeat for each class.

The deck's 4-class confusion matrix (Predicted down the side, Expected across):

	Exp 1	Exp 2	Exp 3	Exp 4
Pred 1	**52**	3	7	2
Pred 2	2	**28**	2	0
Pred 3	5	2	**25**	12

Pred 4	1	1	9	**40**

Class sizes: 1→60, 2→34, 3→43, 4→54.

Collapsing to class-1 vs "NOT class-1"

TP = the 52 on the diagonal. FN = rest of class-1's actual column that got mislabelled (2+5+1 = 8). FP = rest of the "Predicted 1" row (3+7+2 = 12). TN = everything else (119). This gives the deck's reduced matrix: **TP 52, FP 12, FN 8, TN 119**.

B.5 Averaging across classes — Macro vs Micro vs Weighted

With per-class scores, you need one overall number. Three ways, all testable:

Class	Precision %	Recall %	F1 %
1	81.25	86.67	83.87
2	87.50	82.35	84.85
3	56.82	58.14	57.47
4	78.43	74.07	76.19

Macro average — simple unweighted mean

Plain English: average the per-class scores treating every class as equally important (ignores class size). Good when small classes matter as much as big ones.

Macro Precision = 76.00% | Macro Recall = 75.31% | Macro F1 = 75.60%

Micro average — pool everything first

Plain English: add up all classes' TP, FP, FN into global totals, *then* compute. Big classes dominate. Deck totals:

- Net TP = 52+28+25+40 = **145**
- Net FP = (3+7+2)+(2+2+0)+(5+2+12)+(1+1+9) = **46**
- Net FN = (2+5+1)+(3+2+1)+(7+2+9)+(2+0+12) = **46**

$$\text{Micro Precision} = \frac{145}{145+46} = 75.92\% \qquad \text{Micro Recall} = \frac{145}{145+46} = 75.92\%$$

> **EXAM HOOK — the elegant identity** Because FP and FN totals are equal here, **Micro Precision = Micro Recall = Micro F1 = Accuracy = 75.92%**. This equality (micro-avg collapses to accuracy) is a classic exam line.

Weighted average — mean weighted by class size

Plain English: like macro, but each class's score is weighted by how many samples it has. Balances fairness to small classes with respect for big ones.

$$\text{Weighted Precision} = \frac{81.25(60)+87.50(34)+56.82(43)+78.43(54)}{60+34+43+54} = 76.07\%$$

Weighted Recall = 75.92% | Weighted F1 = 75.93%

EXAM HOOK — Macro vs Micro vs Weighted

- **Macro** = simple mean; every class equal; small classes get a loud voice.
- **Micro** = pool TP/FP/FN globally; big classes dominate; equals accuracy in single-label problems.
- **Weighted** = mean weighted by class size (support).

B.6 The ROC Curve & AUC

Plain English: Every classifier has a **threshold** — a cutoff probability above which it says "positive" (default 0.5). Slide that threshold and the trade-off shifts: a low threshold catches more positives (high recall) but raises false alarms; a high threshold is stricter. The **ROC curve** plots this trade-off across *all* thresholds at once.

Axes:

- **Y-axis = True Positive Rate** (= Recall = Sensitivity) = TP/(TP+FN)
- **X-axis = False Positive Rate** = FP/(FP+TN) = 1 − Specificity

> **ANALOGY** — A volume knob for "suspicion." Turn sensitivity all the way up and you flag everyone (catch every real case but also everyone innocent). Turn it down and you flag no one. ROC traces every setting of the knob, showing the full menu of trade-offs.

AUC — Area Under the Curve

Plain English: squash the whole ROC curve into one number — the area beneath it. It's the probability that the model ranks a random positive higher than a random negative.

- **AUC = 1.0** — perfect classifier.
- **AUC = 0.5** — useless; no better than a coin flip (the diagonal "chance level" line).
- **AUC < 0.5** — worse than random (predictions inverted).
- Higher = better; closer to the top-left corner = better.

The deck's Iris ROC (one-vs-rest) illustrates real values: **setosa AUC = 0.89** (easy — linearly separable), **versicolor AUC = 0.66** (hard), **virginica AUC = 0.78**; micro-avg 0.77, macro-avg 0.78. The chance-level diagonal sits at AUC = 0.5.

> **EXAM HOOK — ROC/AUC essentials**
>
> - ROC axes: Y = **TPR (Recall/Sensitivity)**, X = **FPR (1 − Specificity)**.
> - AUC = 1 perfect; AUC = 0.5 = chance (diagonal); < 0.5 worse than random.
> - Top-left corner = ideal. AUC is *threshold-independent* — that's its main advantage over accuracy at a single cutoff.

Glossary

Supervised learning
Learning from labelled examples where the correct answer is known.

Root node
The top node of a tree; the first split; sees all data. Only one exists.

Internal / Decision node
A node that asks a question and splits the data further.

Leaf / Terminal node
An endpoint node giving the final prediction; has no children.

Classification tree
Tree predicting a discrete/categorical target.

Regression tree
Tree predicting a continuous numeric target; leaf value = mean of the region.

Impurity
How mixed the classes are in a node. Zero = pure (one class only).

Entropy
A measure of randomness/disorder; 0 (pure) to 1 (50/50) for two classes.

Information Gain
Parent entropy − weighted child entropy; the tree maximises it to pick splits.

Gini Impurity
$1 - \Sigma p^2$; probability of mislabelling a random pick; faster than entropy.

CART / CHAID
CART uses Gini (classification & regression); CHAID uses chi-square.

Gain Ratio
Normalised Information Gain that penalises many tiny splits.

ANOVA (in trees)
Regression-tree criterion; minimises within-node variance.

RSS / SSE
Residual/Sum of Squared Errors; regression trees minimise this.

MSE
Mean Squared Error = SSE / n.

Overfitting
Model memorises training noise; high variance; fails on new data.

Pre-pruning
Stop growing early (minsplit, minbucket, maxdepth).

Post-pruning
Grow fully, then trim branches using a test set.

minsplit

Min observations in a parent to allow a split (R default 20).

minbucket

Min observations allowed in a leaf.

maxdepth

Maximum tree height (R default 30; deck example 5).

Axis-parallel split

A cut on one variable (horizontal/vertical); trees can only do these.

Ensemble

Combining many models to beat any single one.

Bagging

Bootstrap Aggregating: train trees on with-replacement samples, then vote/average; uses all predictors.

Random Forest

Bagging + random feature subset at each split; m = √K (classification), K/3 (regression).

OOB error

Out-of-Bag: error on records left out of a tree's sample; free validation.

Gradient Boosting

Sequential trees, each correcting prior errors (vs RF's parallel voting).

Confusion matrix

2×2 (or n×n) grid of TP, TN, FP, FN.

True Positive (TP)

Predicted positive, actually positive.

True Negative (TN)

Predicted negative, actually negative.

False Positive (FP)

Predicted positive, actually negative — "false alarm" — Type I error.

False Negative (FN)

Predicted negative, actually positive — "miss" — Type II error.

Accuracy

(TP+TN)/total. Can mislead on imbalanced data.

Precision

TP/(TP+FP). Of predicted positives, how many are right.

Recall / Sensitivity

TP/(TP+FN). Of actual positives, how many found.

Specificity

TN/(TN+FP). Of actual negatives, how many found.

F1-Score

Harmonic mean of precision & recall; rewards balance.

Macro avg

Simple mean across classes; every class equal.

Micro avg

Pool TP/FP/FN globally; equals accuracy in single-label tasks.

Weighted avg

Mean weighted by class size (support).

One-vs-all

Treat one class as positive, the rest as negative, to reuse 2×2 metrics.

ROC curve

Plot of TPR vs FPR across all thresholds.

AUC

Area Under ROC; 1 perfect, 0.5 chance, <0.5 worse than random.

Threshold

Cutoff probability for calling "positive" (default 0.5).

Quick-Revision Cheat Sheet

Decision Trees — core

- **Nodes:** Root (first, all data, one) → Internal/Decision → Leaf/Terminal (prediction, no children).
- **Target:** discrete → Classification tree; continuous → Regression tree. Trees do *both*.
- **Good split** = children as pure as possible (all-positive or all-negative).

Impurity formulas

- $$Entropy = \sum -p(c_i)\log_2 p(c_i) \quad (0\text{ pure} \to 1\text{ at }50/50)$$
- $$Gini = 1 - \sum p^2(c_i) \quad (0\text{ pure} \to 0.5\text{ at }50/50)$$
- $$\text{Info Gain} = \text{Entropy(parent)} - \text{weighted Entropy(children)} \;\rightarrow\; \textbf{maximise}$$
- Gini favours large partitions (squared prob); Entropy favours small (log). Gain Ratio penalises tiny splits. ANOVA = regression.

Deck numbers to remember

- Info-gain example: parent 0.996, children 0.787 & 0.391, weighted 0.615, **gain = 0.38**.
- Gini: **Patrons gain 0.28** beats **Type gain 0** → root = Patrons. (Full node Gini 0.44, weighted 0.22.)
- Regression tree: splits 6.75/7.5/8.25 give SSE 172.67 / 134.5 / **50** → split at **8.25**. Root MSE = 44.19.

Overfitting & pruning

- Deep tree = memorises noise = **high variance = overfit**.
- **minsplit** = min obs in parent to split (default 20). **minbucket** = min obs in leaf. **maxdepth** = height cap (default 30, deck 5).
- Pre-prune = stop early; Post-prune = grow full then trim with a test set.
- Headline weakness: only **axis-parallel** (straight, one-variable) cuts → clumsy on diagonal/curved boundaries.

Ensembles

- **Bagging** = with-replacement samples + all predictors + vote/mean.
- **Random Forest** = Bagging + random feature subset. m = √K (class), K/3 (regr). Bagging = RF with m = K.
- **OOB** = free validation from left-out rows.

- RF = parallel + vote; **Boosting** = sequential + error-correcting.

Confusion matrix & metrics

- TP/TN correct; **FP = Type I (false alarm)**; **FN = Type II (miss)**.
- $$\text{Accuracy}=\frac{TP+TN}{\text{total}} \quad \text{Precision}=\frac{TP}{TP+FP} \quad \text{Recall}=\frac{TP}{TP+FN}$$
- $$\text{Specificity}=\frac{TN}{TN+FP} \quad F1 = 2\cdot\frac{P\cdot R}{P+R}\;(\text{harmonic mean})$$
- Cat/dog deck: Acc 0.80, Dog P/R/F1 = 0.83, Not-Dog = 0.75.
- Recall matters for cancer screening; Precision matters for spam/conviction.

Multi-class averaging

- **Macro** = simple mean (76.00 / 75.31 / 75.60).
- **Micro** = pool globally → Net TP 145, FP 46, FN 46 → **75.92% = Accuracy** (P = R = F1).
- **Weighted** = by class size (76.07 / 75.92 / 75.93).

ROC / AUC

- Y = TPR (Recall); X = FPR (1 − Specificity). Slides over all thresholds.
- AUC: 1 perfect, 0.5 chance (diagonal), <0.5 worse than random. Top-left = best.
- Iris deck: setosa 0.89 (easy), versicolor 0.66 (hard), virginica 0.78.
- AUC's advantage: **threshold-independent.**